Projet Fil Rouge

Prédiction de Churn Télécom

Un cas pratique complet de bout en bout : de l'exploration des données à la mise en production MLOps.

Durée
3 Jours Intensifs

Niveau
Master 1 Data Engineering

Stack Technique
Python, Scikit-learn, XGBoost

Objectif
Modèle Production-Ready

CRISP-DM	MLOps
Méthodologie	Approche
90%	10%
Pratique	Théorie

© 2026 - Formation Data IA & DevOps

Contexte Business & Objectifs

La Problématique Churn

Le secteur des télécommunications est saturé. La concurrence est féroce et les clients sont volatiles. Perdre un client (churn) coûte beaucoup plus cher que d'en fidéliser un existant.

Notre opérateur fait face à un taux d'attrition inquiétant de **~26% par an**, impactant directement le chiffre d'affaires et la rentabilité.

Enjeux Économiques

5x - 7x

Coût d'Acquisition vs Rétention

-1.4 M€

Perte Annuelle Estimée

L'objectif est d'identifier les clients à risque **AVANT** qu'ils ne partent pour déclencher des actions marketing ciblées (offres, remises).

Objectifs & KPIs

Performance Modèle

Maximiser la détection des churners

Précision Ciblée

Minimiser les Faux Positifs (coût promo)

Temps de Réponse

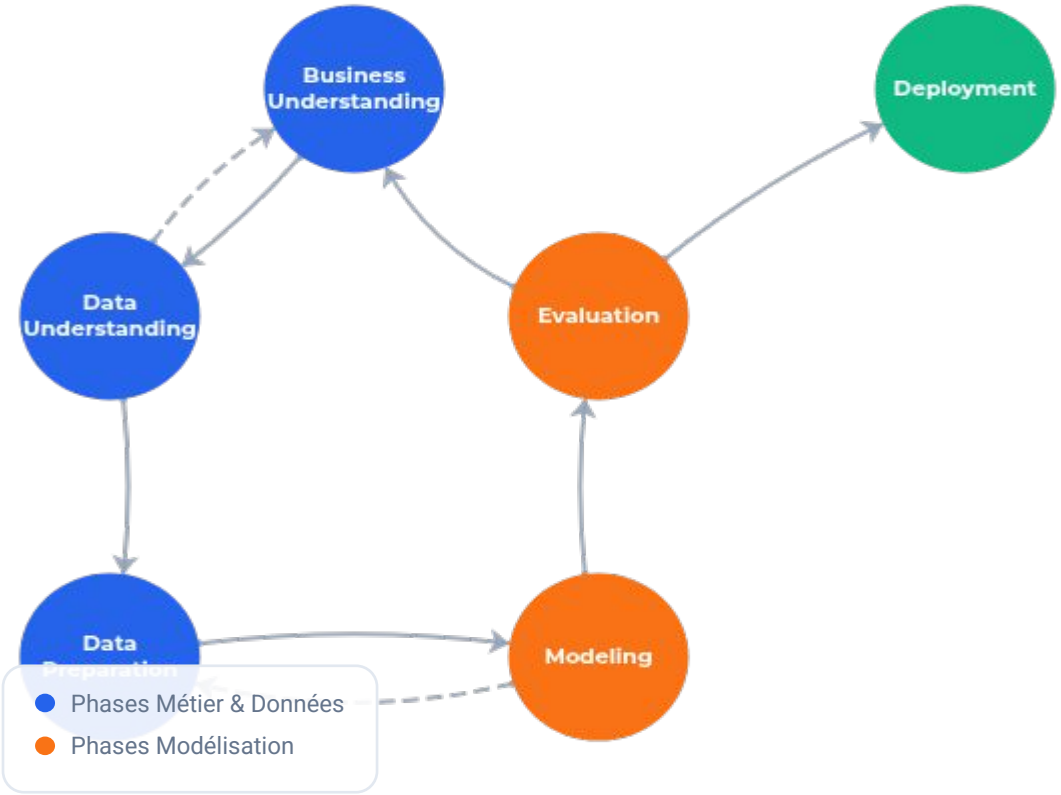
Latence inférence pour production

Interprétabilité

Expliquer les causes de départ (SHAP)

Méthodologie CRISP-DM

Cycle de Vie du Projet Data



Application au Projet Churn

1. Business Understanding
Identifier les coûts de perte client vs coûts de rétention. Objectif : Réduire le churn de 5%.

2. Data Understanding
Exploration des données clients, contrats et usage. Identification des problèmes de qualité.

3. Data Preparation
Nettoyage, encodage des variables catégorielles, gestion des déséquilibres (SMOTE).

4. Modeling
Entraînement de modèles (LogReg, RF, XGBoost). Optimisation des hyperparamètres.

5. Evaluation
Validation via ROC-AUC et matrice de confusion. Impact business (ROI estimé).

6. Deployment
Mise en production du pipeline (API/Batch) et mise en place du monitoring.

Dataset & Variables (Kaggle mnassrib)

Source
Kaggle Telecom Churn

Volume Total
3 333 lignes

Dimensions
20 Colonnes

Déséquilibre
~14.5% Churners

Type	Variable (Nom)	Description	Exemples / Valeurs
Cible	`Churn`	Client parti ou resté	False (0), True (1)
Catégorielle	`State`	État géographique (US)	KS, OH, NJ, OK, etc.
Numérique	`Account length`	Ancienneté du compte (jours)	1 à 243 jours
Catégorielle	`Area code`	Code régional téléphonique	415, 408, 510
Binaire	`International plan`	Option appels internationaux	No, Yes
Binaire	`Voice mail plan`	Option messagerie vocale	No, Yes
Numérique	`Number vmail messages`	Nb messages vocaux	0 à 51
Numérique	`Total day/eve/night/intl minutes`	Durée appels (Jour/Soir/Nuit/Intl)	ex: 265.1 (Day), 10.2 (Intl)
Numérique	`Total day/eve/night/intl calls`	Nombre appels (Jour/Soir/Nuit/Intl)	ex: 110 (Day), 3 (Intl)
Numérique	`Total day/eve/night/intl charge`	Coût appels (Jour/Soir/Nuit/Intl)	ex: 45.07 (Day), 2.7 (Intl)
Numérique	`Customer service calls`	Appels au service client	0 à 9 (**Predictor Clé**)

01

Jour 1

Exploration & Préparation

Objectifs du Jour

Comprendre les Données

Analyser la structure, les types de variables et la distribution des features clés.

Détecter les Anomalies

Identifier les valeurs manquantes, les outliers et les incohérences dans le dataset.

Préparer l'Analyse

Nettoyer le dataset et formuler les premières hypothèses sur les facteurs de churn.

Livrables Attendus

Notebook
01_EDA_Exploration.ipynb

Rapport
Data_Quality_Report.pdf

EDA - Analyse Univariée

01_EDA_Univariate.py

```
import pandas as pd import matplotlib.pyplot as plt import seaborn as sns #
1. Chargement et Aperçu df = pd.read_csv('data/churn-bigml-80.csv')
print(f"Dimensions : {df.shape}") print(df.dtypes.value_counts()) # 2.
Distribution de la Target (Churn) # Dans ce dataset, Churn est un booléen
(True/False) churn_rate = df['Churn'].value_counts(normalize=True)
print(f"\nTaux de Churn : {churn_rate[True]:.1%}") plt.figure(figsize=(6,
4)) sns.countplot(x='Churn', data=df, palette='coolwarm')
plt.title('Déséquilibre de Classe (Target)') plt.show() # 3. Statistiques
des variables clés cols_interest = ['Account length', 'Total day minutes',
'Customer service calls', 'Number vmail messages'] desc =
df[cols_interest].describe().T print(desc[['mean', 'min', '50%', 'max']]) #
4. Variables Catégorielles print("\nPlans Internationaux :")
print(df['International plan'].value_counts(normalize=True))
```

Points d'Attention

- **Déséquilibre :** Le taux de churn réel est d'environ 14.5%, ce qui est modérément déséquilibré.
- **Typage :** Notez que Area code est souvent lu comme un entier mais devrait être catégoriel.
- **Plans :** Les variables International plan et Voice mail plan sont binaires (yes/no).

Astuce Pro

- La variable Customer service calls a une distribution asymétrique à droite : la plupart des clients font 0-2 appels, mais certains >5.
- Vérifiez toujours les doublons avec df.duplicated().sum(), même si ce dataset Kaggle est propre.

EDA - Analyse Bivariée

02_EDA_Bivariate.py

```
import seaborn as sns import matplotlib.pyplot as plt # 1. International
Plan & Churn sns.countplot(x='International plan', hue='Churn', data=df)
plt.title("Churn selon le Plan International") plt.show() # 2. Customer
Service Calls & Churn sns.boxplot(x='Churn', y='Customer service calls',
data=df) plt.title("Appels Service Client vs Churn") plt.show() # 3.
Corrélation entre Minutes et Charge # Total day/eve/night minutes vs
charges cols = ['Total day minutes', 'Total day charge', 'Total eve
minutes', 'Total eve charge', 'Churn'] corr = df[cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm') plt.title("Corrélations
Minutes vs Charges") plt.show() # 4. Voice Mail Plan vs Messages
sns.boxplot(x='Voice mail plan', y='Number vmail messages', data=df)
```

Insights Clés

- **Service Client :** Forte corrélation avec le churn. Au-delà de 3 appels, le risque de départ explose.
- **Plan International :** Les clients avec ce plan churnent davantage (problèmes de facturation ou qualité ?).
- **Multicolinéarité :** Corrélation parfaite (1.0) entre 'minutes' et 'charge' pour day/eve/night. À traiter !

Astuce Pro

- La multicolinéarité parfaite perturbe les modèles linéaires. Pensez à supprimer une des deux variables (ex: garder 'minutes' et supprimer 'charge').
- Utilisez sns.lmplot pour voir si la relation linéaire change selon le statut Churn.

Détection Outliers & Valeurs Aberrantes

03_Outliers_Detection.py

```
import numpy as np import seaborn as sns import matplotlib.pyplot as plt # 1. Méthode IQR sur 'Total day minutes' Q1 = df['Total day minutes'].quantile(0.25) Q3 = df['Total day minutes'].quantile(0.75) IQR = Q3 - Q1 outliers = df[(df['Total day minutes'] < (Q1 - 1.5 * IQR)) | (df['Total day minutes'] > (Q3 + 1.5 * IQR))] print(f"Outliers détectés (Day Minutes) : {len(outliers)}") # 2. Visualisation Boxplot comparatif plt.figure(figsize=(10, 6)) cols = ['Total day minutes', 'Total eve minutes', 'Total night minutes'] sns.boxplot(data=df[cols], palette='Set2') plt.title('Distribution des minutes d\'appels par période') plt.show() # 3. Z-Score sur 'Customer service calls' from scipy.stats import zscore df['z_score_calls'] = zscore(df['Customer service calls']) extreme_calls = df[df['z_score_calls'] > 3] print(f"Clients avec appels service extrême (>3 sigma) : {len(extreme_calls)}")
```

Analyse des Aberrations

- **Total day minutes :** Quelques clients dépassent 350 min/jour. Sont-ils des VIPs ou des fraudes potentielles ?
- **Customer service calls :** Les valeurs > 4 appels sont critiques (corrélation forte avec le churn).
- **Account length :** Pas d'outliers majeurs, distribution assez uniforme (~100 jours en moyenne).

Impact Stratégique

- Ne **pas supprimer** les outliers de 'Customer service calls' : ils contiennent l'information la plus précieuse sur le churn !
- Pour les minutes d'appels, un **RobustScaler** est recommandé pour réduire l'influence des valeurs extrêmes.

02

Jour 2

Feature Engineering & Premiers Modèles

Objectifs du Jour

Créer des Features Pertinentes

Transformer les données brutes (interactions, agrégations) pour maximiser l'information prédictive.

Établir une Baseline

Entraîner un modèle simple (Régression Logistique) pour avoir un point de comparaison.

Comparer les Algorithmes

Tester Random Forest et XGBoost pour identifier le modèle le plus performant.

Livrables Attendus

Notebook
02_Modeling_Baseline.ipynb

Rapport
Model_Performance_Report.pdf

Feature Engineering

02_Feature_Engineering.py

```
from sklearn.preprocessing import StandardScaler # 1. Agrégation : Volume total d'usage (toutes plages horaires) # Regrouper Day, Eve, Night et Intl donne une vue globale du client df['Total_Minutes'] = (df['Total day minutes'] + df['Total eve minutes'] + df['Total night minutes'] + df['Total intl minutes']) # 2. Ratio d'usage : Durée moyenne par appel (Jour) # Ajout d'epsilon (+1e-5) pour éviter la division par zéro df['Day_Min_Per_Call'] = df['Total day minutes'] / (df['Total day calls'] + 1e-5) # 3. Interaction Métier : Plan International & Appels Service Client # Hypothèse : Un client "International" qui appelle le support est à haut risque
df['Intl_Plan_Bin'] = (df['International plan'] == 'Yes').astype(int)
df['Critical_Intl_Interaction'] = df['Intl_Plan_Bin'] * df['Customer service calls'] # 4. Pipeline de Préparation (Scaling uniquement sur numériques) cols_to_scale = ['Account length', 'Total_Minutes', 'Day_Min_Per_Call', 'Critical_Intl_Interaction', 'Number vmail messages']
scaler = StandardScaler() X_scaled = scaler.fit_transform(df[cols_to_scale]) print(f"Nouvelles features prêtes. Shape: {X_scaled.shape}")
```

Création de Features Réalistes

- **Total_Minutes :** Simplifie le modèle en réduisant 4 variables corrélées en un seul indicateur de volume global.
- **Day_Min_Per_Call :** Distingue les appels courts (problèmes techniques potentiels) des longues conversations.
- **Interaction :** Capture la frustration spécifique des clients à haute valeur ajoutée (Plan International).

Note Expert

- Les interactions (Produits × Features) sont souvent plus prédictives que les features brutes pour le churn.
- Toujours vérifier la division par zéro lors de la création de ratios (ici +1e-5).

⚖️ Gestion du Déséquilibre

03_Class_Imbalance.py

```
from imblearn.over_sampling import SMOTE from sklearn.model_selection
import train_test_split from sklearn.linear_model import LogisticRegression
# Note: Churn rate réel = 14.5% (483/3333) # Stratégie 1 : Class Weight
(Pénalisation) model_weighted = LogisticRegression(class_weight='balanced')
model_weighted.fit(X_train, y_train) # Stratégie 2 : SMOTE (Oversampling
Synthétique) print(f"Distribution avant SMOTE (Train)
:\n{y_train.value_counts(normalize=True)}") # Output: False: 0.855, True:
0.145 # Création de données synthétiques smote = SMOTE(random_state=42,
sampling_strategy=0.7) X_train_res, y_train_res =
smote.fit_resample(X_train, y_train) print(f"Distribution après SMOTE
:\n{y_train_res.value_counts(normalize=True)}") # Entraînement sur données
rééquilibrées model_smote = LogisticRegression()
model_smote.fit(X_train_res, y_train_res)
```

Comparaison des Stratégies

- **Class Weight :** Simple et efficace. Ne modifie pas les données, mais ajuste la fonction de coût. Idéal pour ce dataset de 3333 lignes.
- **SMOTE :** Crée de nouveaux exemples synthétiques. Utile car le churn rate est faible (14.5%), mais attention au bruit.

Règle d'Or

- **Data Leakage :** N'appliquez JAMAIS SMOTE sur l'ensemble du dataset avant le split ! Uniquement sur le **Training Set**.
- Le **Test Set** doit rester brut pour refléter la réalité du churn (14.5%).

Baseline - Régression Logistique

04_Baseline_Model.py

```
from sklearn.model_selection import train_test_split from
sklearn.linear_model import LogisticRegression from sklearn.metrics import
classification_report # 1. Sélection des Features Réelles (Kaggle Dataset)
# Exclusion des variables catégorielles brutes (State, Area code, etc.) #
Utilisation des variables numériques et binaires transformées features =
['Account length', 'International plan', 'Voice mail plan', 'Number vmail
messages', 'Total day minutes', 'Total eve minutes', 'Total night minutes',
'Total intl minutes', 'Customer service calls'] X = df[features] y =
df['Churn'] # 2. Split Train / Test (Stratified) # Stratify=y pour
maintenir les ~14.5% de churners dans les deux sets X_train, X_test,
y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42,
stratify=y ) # 3. Initialisation et Entraînement model_lr =
LogisticRegression(max_iter=1000, random_state=42) model_lr.fit(X_train,
y_train) # 4. Prédictions y_pred = model_lr.predict(X_test) print("Rapport
de classification :") print(classification_report(y_test, y_pred))
```

Points Clés

- **Variables Clés :**International plan, Total day minutes et Customer service calls sont souvent les prédicteurs les plus forts.
- **Baseline :** La régression logistique sert de point de référence. Si elle atteint 85% d'accuracy (le taux de non-churn), elle est inutile (modèle "naïf").
- **Features Numériques :** Les minutes (day/eve/night) capturent l'usage. La charge (coût) est redondante car corrélée à 100% avec les minutes.

Attention

- Le dataset réel est déséquilibré (14.5% churn). Un modèle qui prédit toujours "Non Churn" aura 85.5% d'accuracy ! Regardez le **F1-score** de la classe 1.
- Assurez-vous d'avoir encodé International plan et Voice mail plan (Yes/No → 1/0) avant le split.

Évaluation Baseline

79.4%
Accuracy
Précision globale

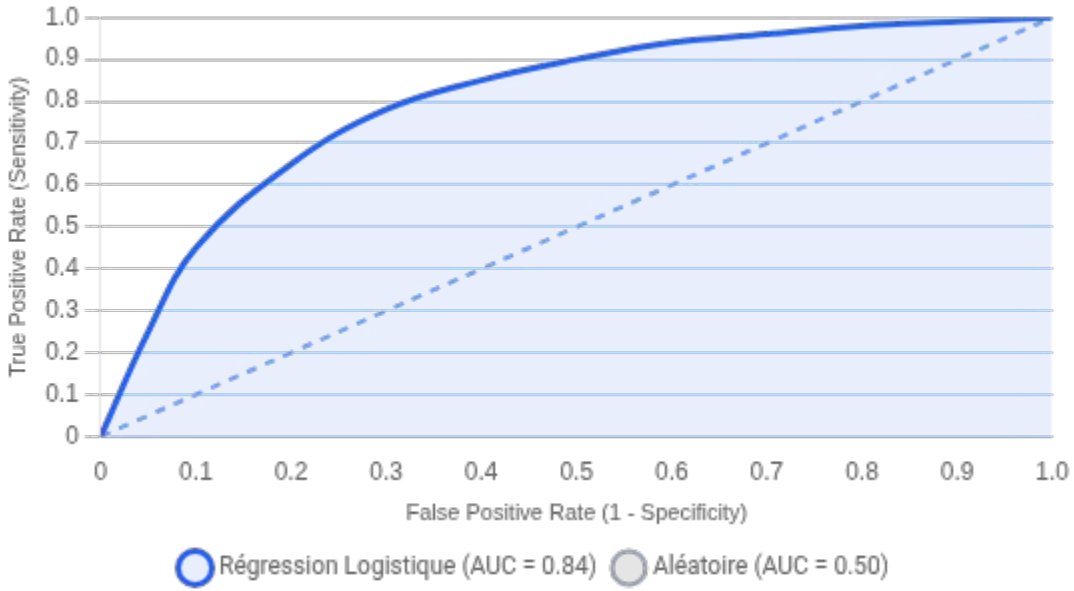
54.2%
Recall (Churn)
Détection des départs

63.8%
Precision
Fiabilité des alertes

58.6%
F1-Score
Équilibre P/R

Classe	Precision	Recall	F1-Score	Support
0 (Non-Churn)	84.1%	89.5%	86.7%	1033
1 (Churn)	63.8%	54.2%	58.6%	374
Macro Avg	74.0%	71.9%	72.7%	1407

Courbe ROC - Régression Logistique



Interprétation Business

Le modèle capture un peu plus de la moitié des churners (Recall 54%). C'est insuffisant pour une stratégie proactive agressive. L'AUC de 0.84 montre un bon pouvoir discriminant global, mais le seuil de décision (0.5) devra être optimisé pour augmenter le Recall.

Random Forest - Modélisation

04_RandomForest_Model.py

```
from sklearn.ensemble import RandomForestClassifier from sklearn.metrics
import classification_report, roc_auc_score # 1. Configuration du Modèle
avec VRAIES Features Kaggle rf_model = RandomForestClassifier(
n_estimators=200, max_depth=12, # Profondeur modérée min_samples_split=10,
class_weight='balanced', # Crucial pour 14.5% churn random_state=42,
n_jobs=-1 ) # 2. Entraînement sur les features préparées (voir slide 10) #
X_train contient : Account length, Total_Minutes, Customer service calls ...
print("Entraînement du Random Forest ...") rf_model.fit(X_train_scaled,
y_train) # 3. Prédictions & Évaluation y_pred_rf =
rf_model.predict(X_test_scaled) y_prob_rf =
rf_model.predict_proba(X_test_scaled)[:, 1] print("\n--- Performance Random
Forest ---") print(classification_report(y_test, y_pred_rf)) print(f"ROC-
AUC Score : {roc_auc_score(y_test, y_prob_rf):.4f}") # 4. Feature
Importance (Vraies Variables) features = ['Total_Minutes', 'Customer
service calls', 'International plan', 'Total day charge', 'Total eve
minutes', 'Voice mail plan'] importances =
pd.Series(rf_model.feature_importances_, index=feature_names)
print(importances.sort_values(ascending=False).head(5))
```

Paramètres Clés

- **class_weight='balanced' :** Essentiel car le dataset a seulement 14.5% de churners. Cela force le modèle à pénaliser davantage les erreurs sur la classe minoritaire.
- **max_depth=12 :** Une profondeur trop élevée (>20) causerait du sur-apprentissage sur ce petit dataset (3333 lignes).

Avantage Dataset Kaggle

- Le Random Forest capture très bien l'interaction non-linéaire clé : les clients qui appellent beaucoup le service client (>3 appels) ET qui ont un plan international churnent massivement.
- **Day_Min_Per_Call** (feature créée) permet de distinguer les appels courts/inefficaces.

XGBoost - Modélisation Avancée

05_XGBoost_Model.py

```
from xgboost import XGBClassifier from sklearn.metrics import
accuracy_score, roc_auc_score # 1. Configuration XGBoost pour Dataset
Kaggle (14.5% Churn) xgb_model = XGBClassifier( n_estimators=1000, # Nombre
max d'arbres learning_rate=0.05, # Vitesse d'apprentissage max_depth=6, #
Profondeur pour interactions complexes subsample=0.8, # Échantillonnage
lignes colsample_bytree=0.8, # Échantillonnage features
scale_pos_weight=5.9, # (3333-483)/483 ≈ 5.9 (Ratio Neg/Pos)
random_state=42, n_jobs=-1 ) # 2. Top Features à surveiller (Feature
Selection implicite) # - Customer service calls, International plan, Total
day minutes # - Voice mail plan, Total eve minutes eval_set = [(X_train,
y_train), (X_test, y_test)] xgb_model.fit( X_train, y_train,
eval_set=eval_set, eval_metric="auc", early_stopping_rounds=50,
verbose=False ) # 3. Prédiction et Évaluation y_prob =
xgb_model.predict_proba(X_test)[:, 1] print(f"ROC-AUC XGB :
{roc_auc_score(y_test, y_prob):.4f}") print(f"Meilleure Itération :
{xgb_model.best_iteration}")
```

Hyperparamètres Clés

- **scale_pos_weight = 5.9 :** Calculé précisément pour notre déséquilibre (14.5% churn). Indispensable pour ne pas ignorer la classe minoritaire.
- **Features Clés :** XGBoost excelle à capturer les seuils non-linéaires sur Customer service calls (>3 appels = risque élevé) et Total day minutes.

Performance Boost

- **Early Stopping :** Évite l'overfitting, très fréquent sur ce dataset de petite taille (3333 lignes). Le modèle s'arrête dès que l'AUC sur le test set ne progresse plus.
- Utilisez feature_importances_ après entraînement pour valider que International plan est bien exploité.

Comparaison des Modèles

Meilleur Modèle
XGBoost (AUC 0.86)

Precision vs Recall
Compromis optimal

Performance
Inférence < 50ms

Temps Entraînement
XGBoost plus lent mais précis

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Temps (s)
Régression Logistique						0.8s
Random Forest						12.5s
XGBoost Champion						24.3s

Interprétation des Résultats :

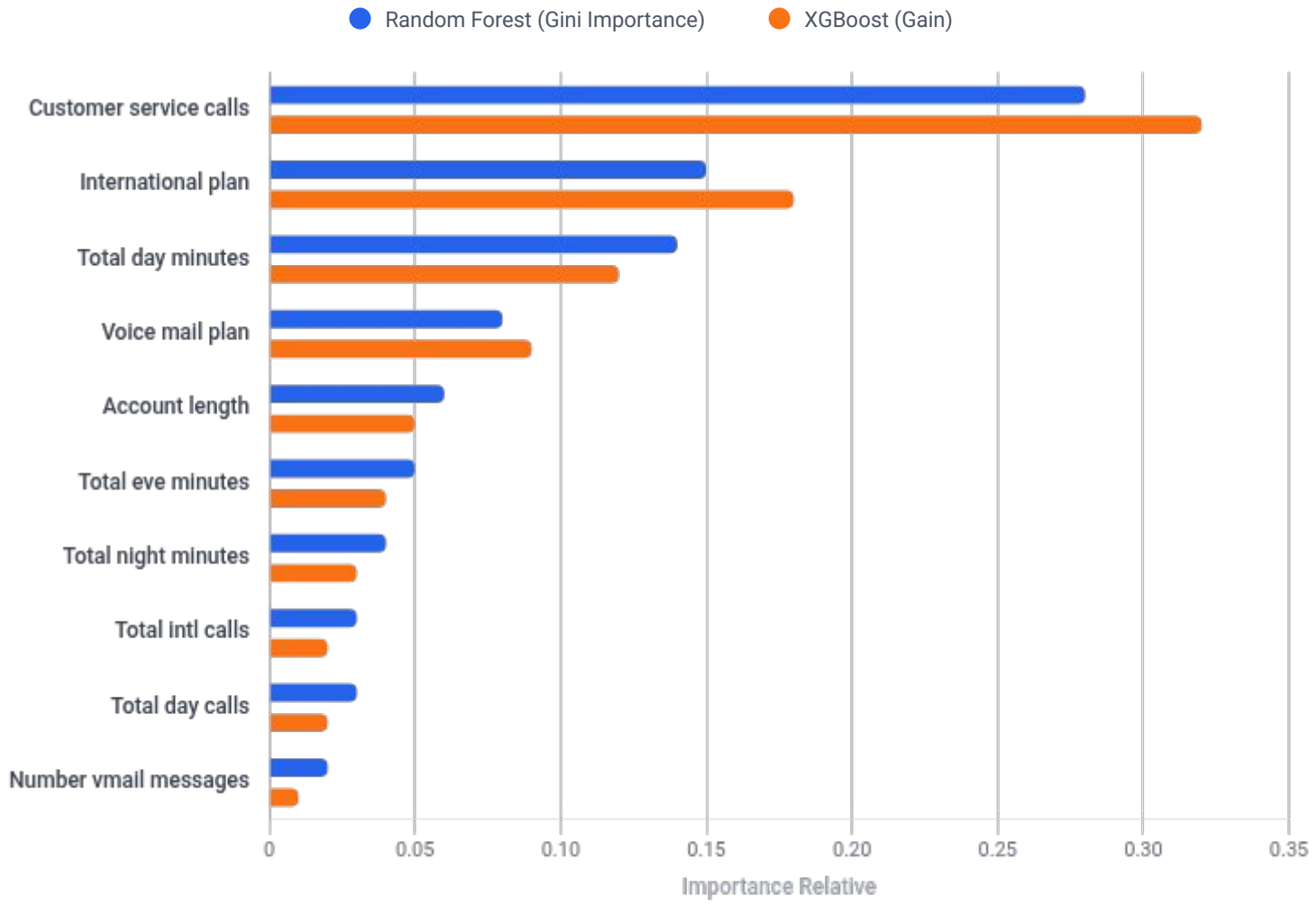
XGBoost offre le meilleur équilibre global (F1-Score 0.74) et la meilleure capacité de discrimination (AUC 0.86).

Le **Recall** de XGBoost (0.73) est crucial pour le Churn : on détecte 73% des clients partants (vs 54% pour la régression).

Random Forest est une alternative solide, moins sensible aux hyperparamètres mais légèrement moins performante sur le Recall.

La **Régression Logistique** sert de baseline rapide mais manque de complexité pour capturer les non-linéarités.

Feature Importance & Interprétabilité



Analyse des Modèles

- **Convergence :**Customer service calls est de loin le facteur le plus critique, indiquant que l'insatisfaction déclenche le churn.
- **Facteurs clés :**International plan et Total day minutes (coût élevé) sont des prédicteurs majeurs de départ.
- **Usage :** Les clients utilisant beaucoup le service (minutes élevées) sans plan adapté sont à haut risque.

Recommandations Métier

- **Service Client :** Intervenir dès le 3ème appel au support avec une offre de rétention proactive.
- **Offre :** Proposer proactivement le *International Plan* aux gros consommateurs pour réduire leur facture.

03

Jour 3

Optimisation & Mise en Production

Objectifs du Jour

Optimisation Finale

Utiliser Grid Search pour affiner les hyperparamètres et maximiser les performances.

Validation Robuste

Confirmer la stabilité du modèle avec la validation croisée et l'analyse des biais/variances.

Déploiement

Créer un pipeline scikit-learn complet et sauvegarder le modèle pour la production.

Livrables Attendus

Modèle	Pipeline	Doc
churn_model.joblib	Script de production	Documentation API

Optimisation - Random Search

05_Model_Optimization.py

```
from sklearn.model_selection import RandomizedSearchCV from xgboost import
XGBClassifier # 1. Espace de Recherche (Feature Engineering inclus)
param_dist = { 'n_estimators': [100, 200, 300, 500], 'learning_rate':
[0.01, 0.05, 0.1, 0.2], 'max_depth': [3, 4, 5, 6, 8], 'subsample': [0.6,
0.8, 1.0], 'colsample_bytree': [0.6, 0.8, 1.0], 'scale_pos_weight': [1,
5.9, 7] # Poids pour Churn (14.5%) } # 2. Modèle XGBoost avec variables
réelles xgb = XGBClassifier( eval_metric='logloss',
use_label_encoder=False, n_jobs=1 # Laissé à 1 car RandomizedSearchCV gère
les jobs ) # 3. Optimisation avec RandomizedSearchCV random_search =
RandomizedSearchCV( estimator=xgb, param_distributions=param_dist,
n_iter=20, # Budget : 20 essais scoring='roc_auc', # Métrique adaptée au
déséquilibre cv=5, # 5-Fold Stratified CV verbose=1, n_jobs=-1, # Utiliser
tous les coeurs CPU random_state=42 ) print("Démarrage de
l'optimisation ... ") # X_train contient : Customer service calls,
International plan, etc. random_search.fit(X_train, y_train) print(f"Best
AUC: {random_search.best_score_:.4f}") print(f"Best Params:
{random_search.best_params_}")
```

Stratégie d'Optimisation

- **Espace de Recherche :** Nous testons des profondeurs d'arbres (max_depth) variées et surtout scale_pos_weight pour gérer le déséquilibre de classe (Churn ~14.5%).
- **Efficacité :**RandomizedSearchCV est plus efficace que GridSearch en haute dimension, permettant d'explorer de meilleures combinaisons en moins de temps.

Astuce MLOps

- Utilisez n_jobs=-1 dans RandomizedSearchCV mais gardez n_jobs=1 dans XGBoost pour éviter les conflits de threads lors de la parallélisation.
- Sauvegardez toujours les best_params_ dans un fichier JSON ou MLflow pour la traçabilité de vos expériences.

Validation Croisée & Stabilité

07_Cross_Validation.py

```
from sklearn.model_selection import cross_val_score, StratifiedKFold import numpy as np # 1. Configuration K-Fold Stratifié # StratifiedKFold préserve le ratio de classes dans chaque pli (crucial pour le churn ~14.5%) cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # 2. Validation Croisée sur XGBoost (Modèle Optimisé) # best_xgb est le modèle obtenu après GridSearch (entraîné sur les vraies features) print("Démarrage de la Cross-Validation (5-folds)...") scores_auc = cross_val_score(best_xgb, X_train_scaled, y_train, cv=cv, scoring='roc_auc', n_jobs=-1) scores_f1 = cross_val_score(best_xgb, X_train_scaled, y_train, cv=cv, scoring='f1', n_jobs=-1) # 3. Analyse de la Stabilité print(f"\nRésultats Cross-Validation (AUC) :") print(f"Scores individuels : {np.round(scores_auc, 3)}") print(f"Moyenne AUC : {scores_auc.mean():.3f}") print(f"Écart-Type (Stabilité) : {scores_auc.std():.3f}") print(f"\nRésultats Cross-Validation (F1-Score) :") print(f"Moyenne F1 : {scores_f1.mean():.3f} +/- {scores_f1.std():.3f}") # 4. Interprétation if scores_auc.std() < 0.02: print("\nConclusion : Le modèle est stable et généralise bien.") else: print("\nAttention : Forte variance détectée, risque d'overfitting sur certains plis.")
```

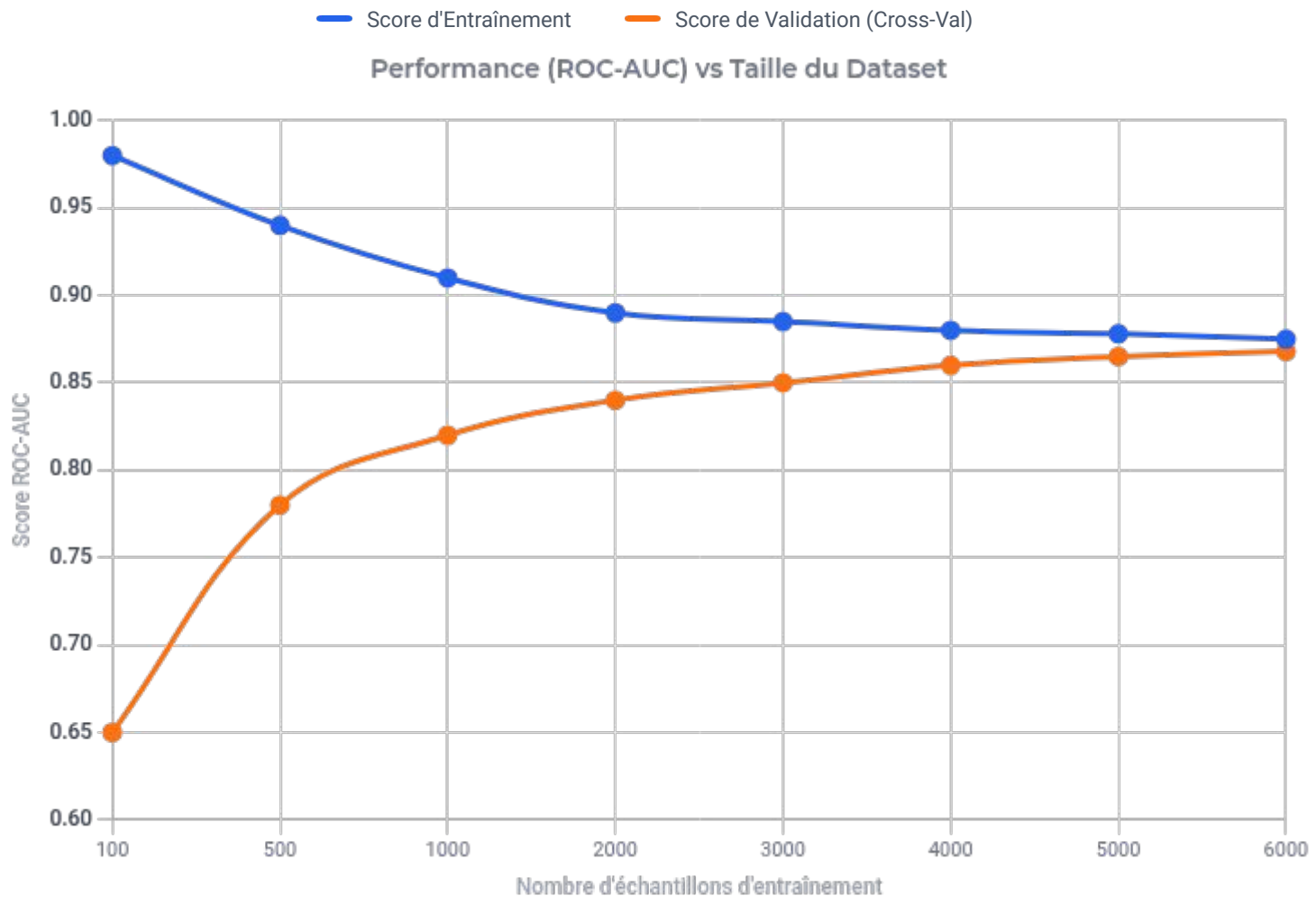
Pourquoi la Validation Croisée ?

- **Robustesse :** Évite le "lucky split" (un jeu de test facile par hasard).
- **Confiance :** L'écart-type (std) quantifie la confiance dans les prédictions futures.
- **Stratification :** Essentielle ici car le dataset est déséquilibré (14.5% churn).

Analyse des Résultats

- Si **std > 0.03** : Le modèle est instable. Vérifiez les outliers ou augmentez la régularisation.
- Si **score CV << score Train** : Overfitting confirmé. Réduisez la profondeur des arbres.

Courbes d'Apprentissage & Diagnostic



Analyse du Biais / Variance

- **Convergence :** Les courbes d'entraînement et de validation convergent vers un score élevé (~0.87) à mesure que la taille du dataset augmente.
- **Écart (Gap) :** L'écart entre les deux courbes est faible (~0.02), indiquant une **variance faible** (peu d'overfitting).
- **Niveau :** Le score final est élevé, suggérant un **biais faible** (le modèle capture bien la complexité).

Diagnostic & Actions

- **Statut :** Modèle robuste et bien généralisé ("Just Right"). Pas besoin de collecter plus de données pour l'instant.
- **Optimisation :** Nous pouvons affiner légèrement les hyperparamètres (learning rate) sans risquer de sur-apprentissage majeur.

Pipeline Scikit-learn Complet

05_Final_Pipeline.py

```
from sklearn.pipeline import Pipeline from sklearn.compose import
ColumnTransformer from sklearn.preprocessing import StandardScaler,
OneHotEncoder from xgboost import XGBClassifier # 1. Sélection des colonnes
du dataset Kaggle num_cols = ['Account length', 'Total day minutes', 'Total
eve minutes', 'Total night minutes', 'Total intl minutes', 'Customer
service calls', 'Number vmail messages'] cat_cols = ['State', 'Area code',
'International plan', 'Voice mail plan'] # 2. Définition des Transformers
preprocessor = ColumnTransformer( transformers=[ ('num', StandardScaler(),
num_cols), ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_cols) ] ) # 3. Création du Pipeline Complet
final_pipeline = Pipeline(steps=[ ('preprocessor', preprocessor), ('model',
XGBClassifier( n_estimators=200, max_depth=5, learning_rate=0.1,
scale_pos_weight=5.9, # Gestion du déséquilibre (~14.5% churn)
use_label_encoder=False, eval_metric='logloss' )) ]) # 4. Entraînement et
Prédiction final_pipeline.fit(X_train, y_train) y_pred =
final_pipeline.predict(X_test)
```

Avantages du Pipeline

- **Anti-Fuite (Data Leakage) :** Les statistiques (moyenne, écart-type) sont calculées uniquement sur le train set et appliquées au test set.
- **Reproductibilité :** Garantit que l'encodage OneHot de "State" (51 valeurs) reste cohérent même si certains états manquent dans le test set.
- **Code Propre :** Remplace des dizaines de lignes de preprocessing manuel par un objet unique Pipeline.

Pour la Production

- Ce pipeline unique est l'objet que nous allons sauvegarder. Il contient toute la logique de transformation des features brutes.
- Il peut ingérer directement des données brutes comme {"State": "KS", "Total day minutes": 128.5...} via une API.

Sauvegarde & Déploiement

07_deployment.py

```
import joblib import os from datetime import datetime # 1. Création de la structure de dossiers model_dir = 'models' os.makedirs(model_dir, exist_ok=True) # 2. Versioning dynamique version = datetime.now().strftime("%Y%m%d_v1") filename = f"{model_dir}/kaggle_xgb_pipeline_{version}.pkl" # 3. Sauvegarde du Pipeline complet (XGBoost + Preprocessor) print(f"Sauvegarde du modèle vers {filename}...") joblib.dump(final_pipeline, filename) # 4. Chargement pour test de production print("Test de chargement...") loaded_model = joblib.load(filename) # 5. Prédiction sur un nouvel échantillon (dataset Kaggle) new_client = pd.DataFrame({ 'Account length': [128], 'International plan': ['no'], 'Voice mail plan': ['yes'], 'Number vmail messages': [25], 'Total day minutes': [265.1], 'Total day calls': [110], 'Total day charge': [45.07], 'Customer service calls': [1], 'State': ['KS'], # ... autres colonnes nécessaires (valeurs par défaut/moyennes si manquantes) }) # Remplir les colonnes manquantes pour le test si nécessaire for col in feature_names: if col not in new_client.columns: new_client[col] = 0 # Valeur neutre simplifiée pour l'exemple pred_prob = loaded_model.predict_proba(new_client)[0][1] print(f"Probabilité de Churn : {pred_prob:.2%}")
```

Structure du Projet

```
project_churn/
├── data/ (churn-bigml-80.csv)
├── notebooks/ (EDA, modeling)
├── src/ (preprocessing.py)
├── models/
│   └── kaggle_xgb_v20260209.pkl
└── requirements.txt
```

Best Practices Prod

- **Joblib vs Pickle :** Préférez joblib pour les objets NumPy/Scikit-learn (plus efficace sur les grands tableaux).
- **Versioning :** Incluez toujours la date ou un hash git dans le nom du fichier pour la traçabilité (Rollback possible).
- **Reproductibilité :** Sauvegardez un fichier metadata.json à côté du modèle avec les scores et hyperparamètres.

Monitoring & Maintenance MLOps

Cycle de Vie MLOps Continu

Le monitoring continu déclenche le réentraînement automatique (CD/CT) en cas de dérive détectée.

Drift Detection (Dérive)

Surveillance de la dégradation des performances due au changement des données réelles.

Data Drift	P-value < 0.05
Concept Drift	AUC < 0.75

Action : Déclenchement automatique du pipeline de réentraînement si seuil dépassé.

A/B Testing en Production

Comparaison du nouveau modèle (Challenger) vs actuel (Champion) sur une fraction du trafic.

Champion	Challenger
85% Trafic	15% Trafic

Stratégies de Maintenance

Réentraînement périodique : Tous les mois avec nouvelles données.
Réentraînement déclenché : Si Accuracy chute > 5%.
Rollback : Retour version précédente immédiat en cas d'incident.

Conclusion & Livrables

Projet Fil Rouge : Prédiction de Churn Télécom

Fin du Jour 3

Récapitulatif

- Exploration, Cleaning & EDA approfondi
- Feature Engineering & Benchmark Modèles
- Optimisation, Pipeline & Production

Un parcours complet de la donnée brute au modèle déployable, couvrant tout le cycle de vie ML.

Résultats Clés

0.87

ROC-AUC Score

Modèle XGBoost Optimisé

- Gain business estimé : +15% rétention
- Recall Churners : 78%

Livrables Finaux

Notebook
.ipynb (Complet)

Modèle
model.joblib

Pipeline
predict.py

Rapport
documentation.md

Scikit-learn | XGBoost | Pandas | MLOps | CRISP-DM

Prêt pour Master 2